Adding Items to an Azure Cosmos DB Container

Summary:-

In this lab, you will create an Azure Cosmos DB database and container, add JSON documents, and query the stored data using SQL-like queries. You will configure a partition key, insert multiple documents, execute queries to retrieve and filter data, and understand how Azure Cosmos DB organizes and searches NoSQL documents.

Let's Start:-

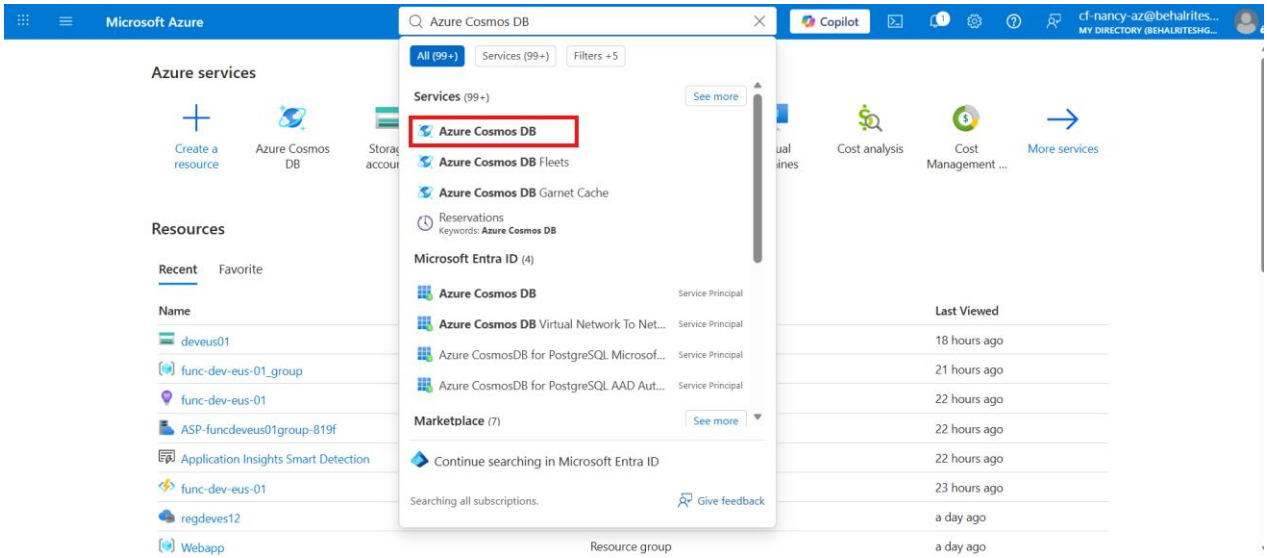
After completing this lab, you will be able to:

- Create an Azure Cosmos DB account and database.
- Create a container with a partition key.
- Add JSON documents to a container.
- Execute SQL queries to retrieve and filter documents.
- Search documents using SQL operators in Azure Cosmos DB.

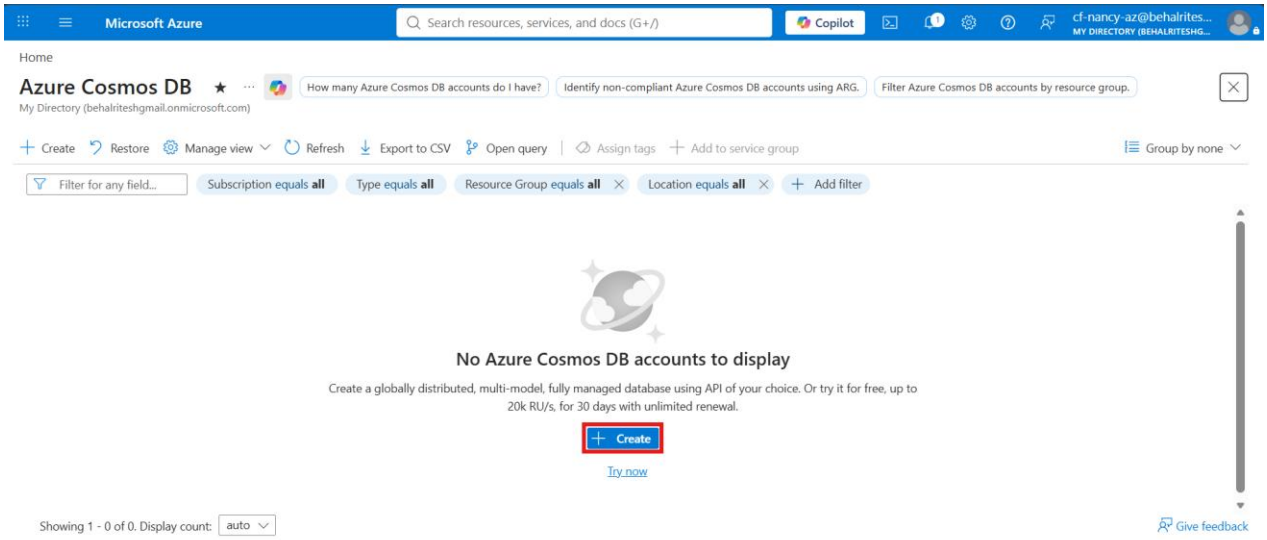
Lab Steps:-

Step 1 – Create an Azure Cosmos DB account.

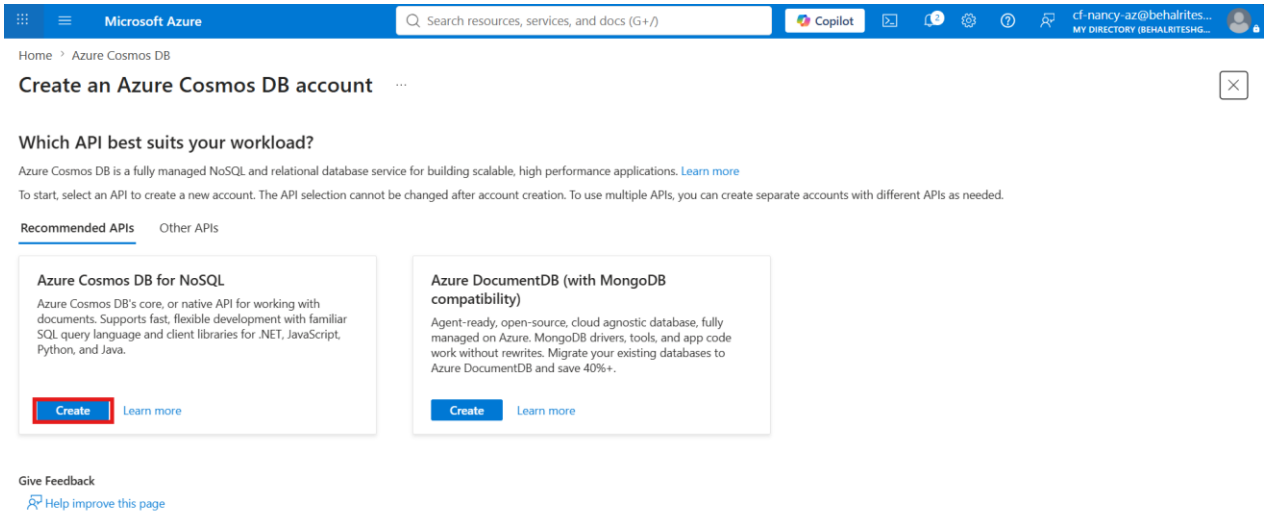
- Sign in to the Azure portal.
- Search for **Azure Cosmos DB**.



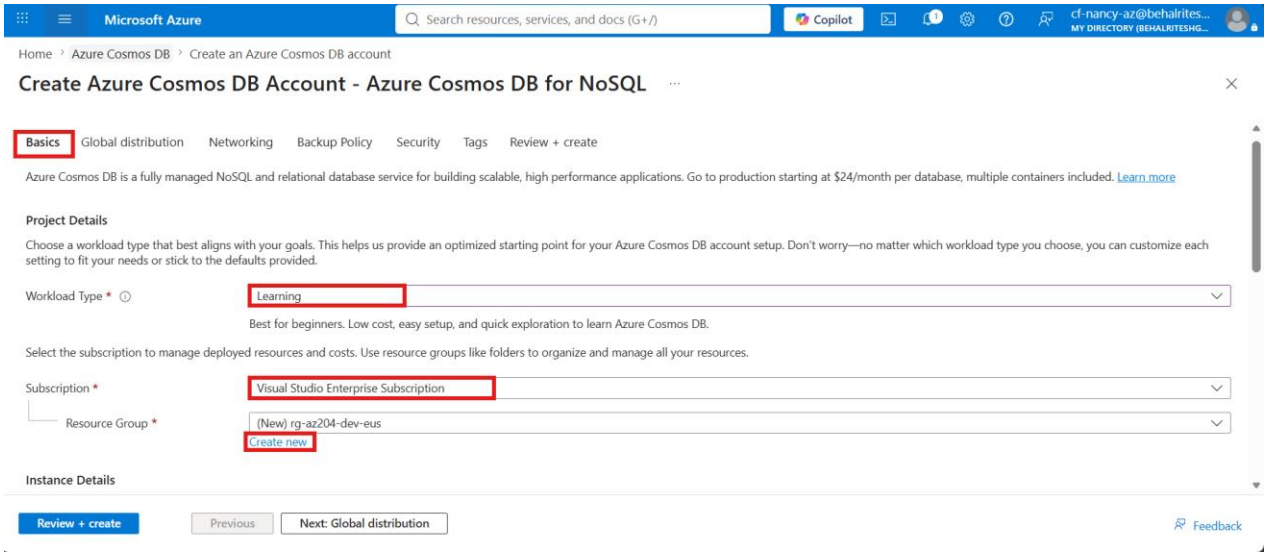
- Select **Azure Cosmos DB.** Click Create.



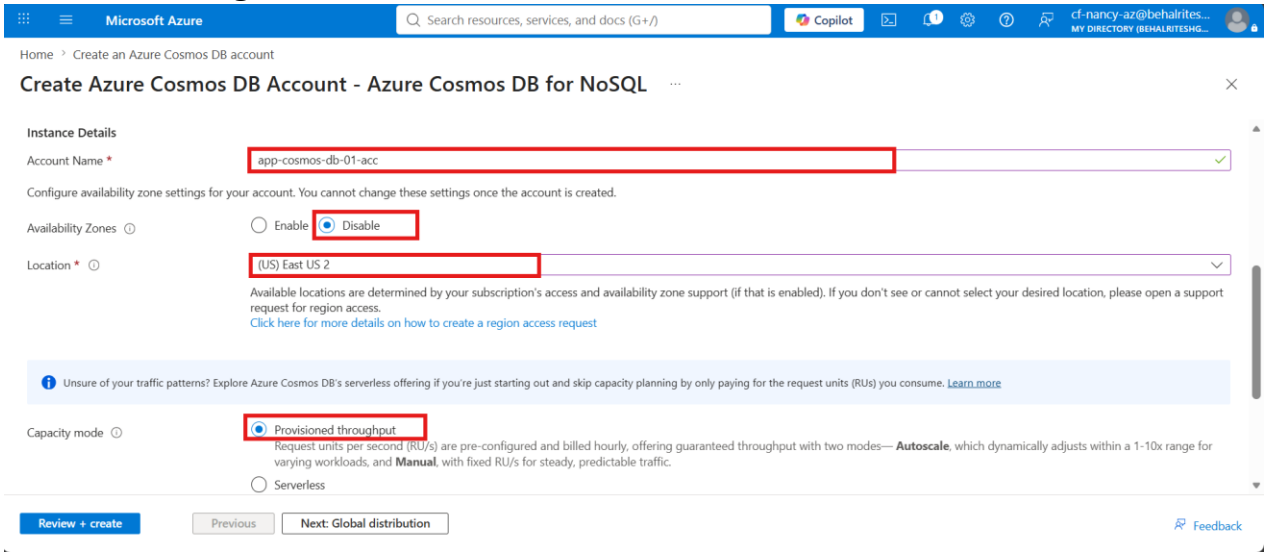
- Under **Recommended**, select **Azure Cosmos DB for NoSQL**.



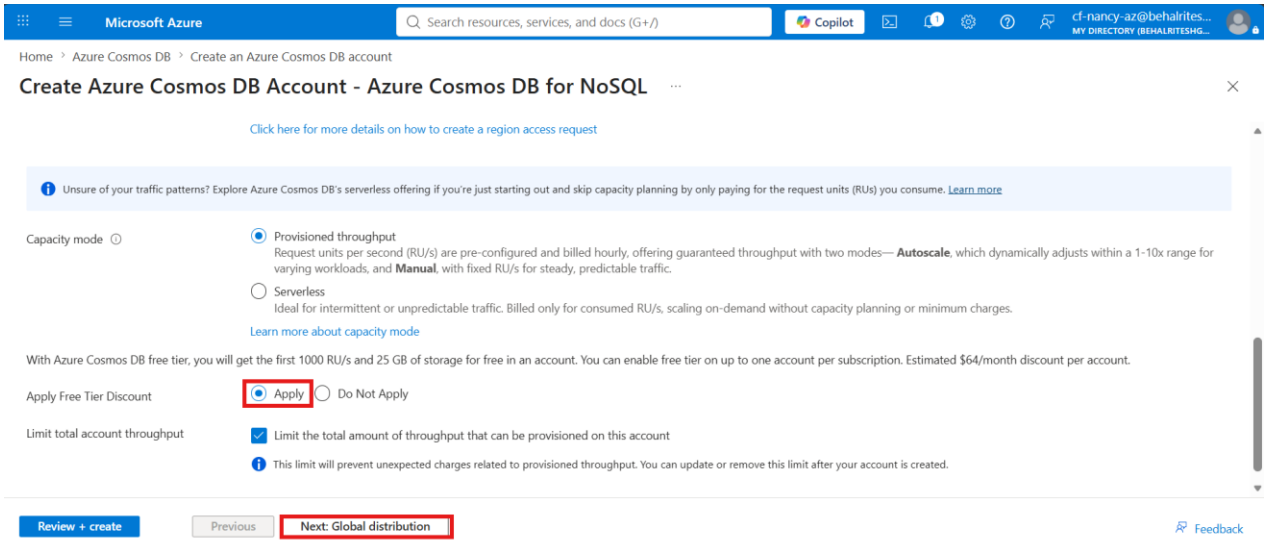
- Click **Create**.
- Configure the following settings:
  - **Workload Type:** Learning
  - **Subscription:** Select your Azure subscription.
  - **Resource Group:** Select your existing resource group or create a new.



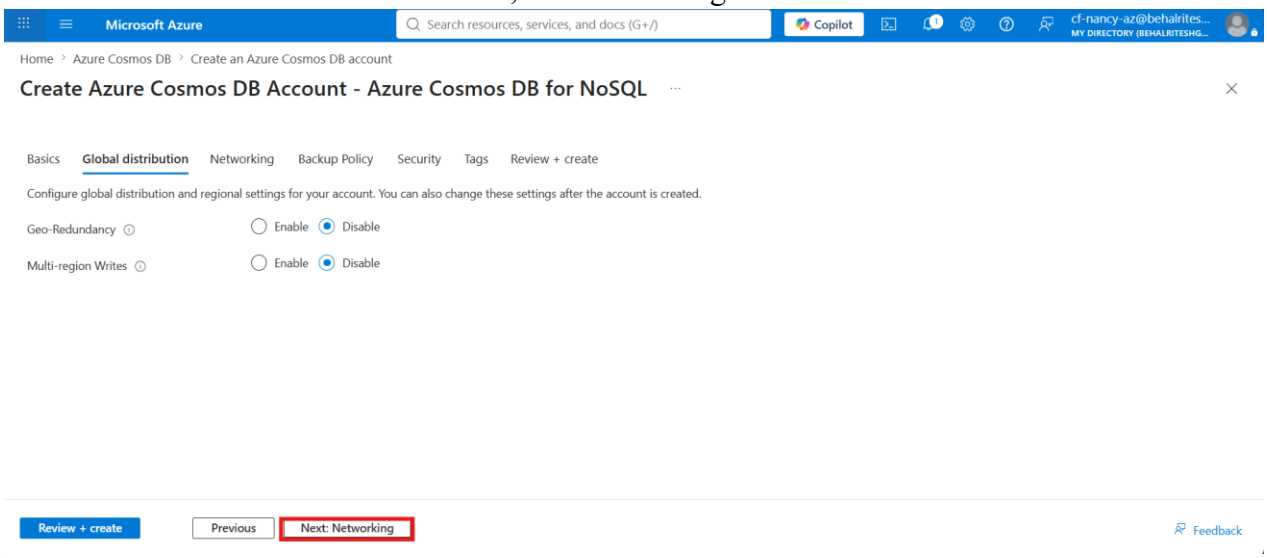
- **Account Name:** Enter a unique name.
- **Region:** Select East US/ East US 2.



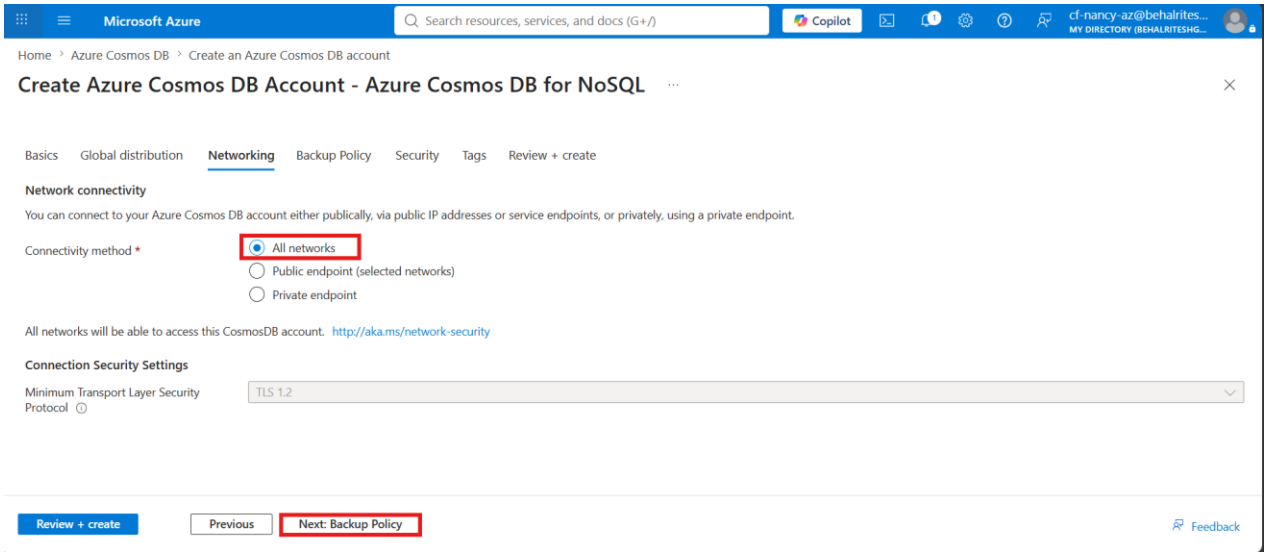
- Under **Capacity Mode**, select **Provisioned Throughput**.
- Enable **Apply Free Tier Discount**.



- Click **Next**.
- On the **Global Distribution** tab, leave all settings at their default values.



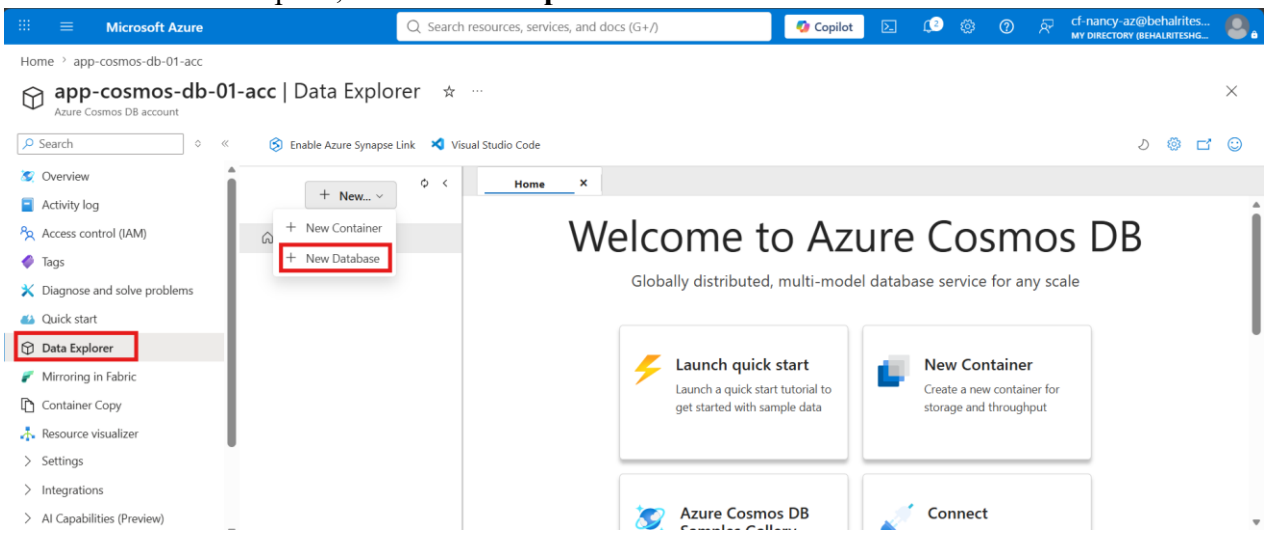
- Click **Next**.
- On the **Networking** tab:
    - Select **Enable access from all networks**.



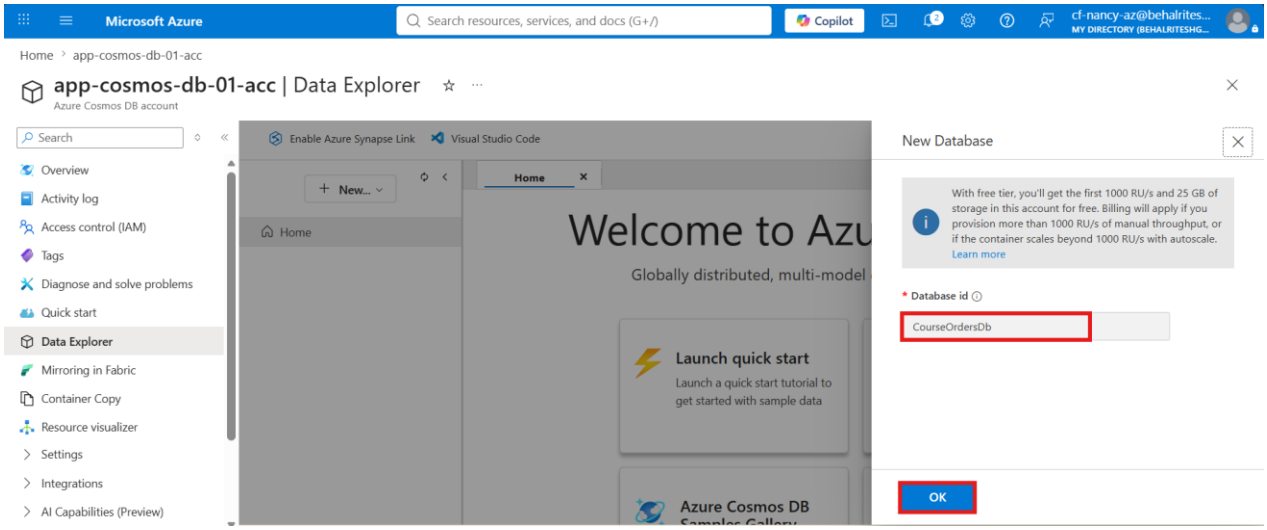
- Leave all settings as default and Click **Review + create**.
- This step created an Azure Cosmos DB account.

Step 2 – Create a Database

- From the left pane, select **Data Explorer**.



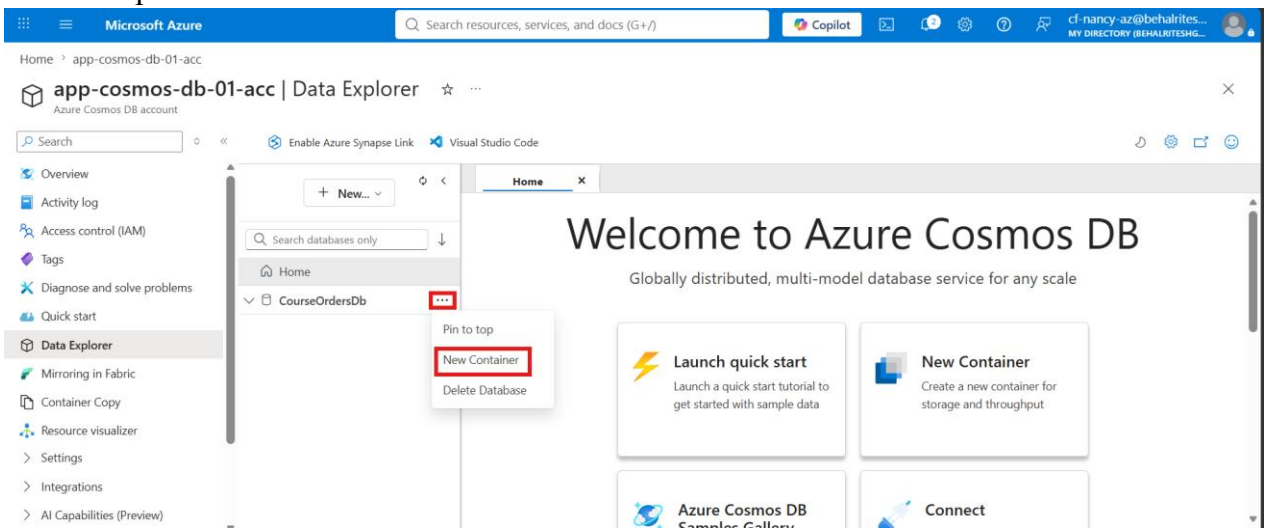
- Under the **NoSQL API**, click **New Database**.
- Enter the following details:
  - **Database ID:** CosmosOrdersDb



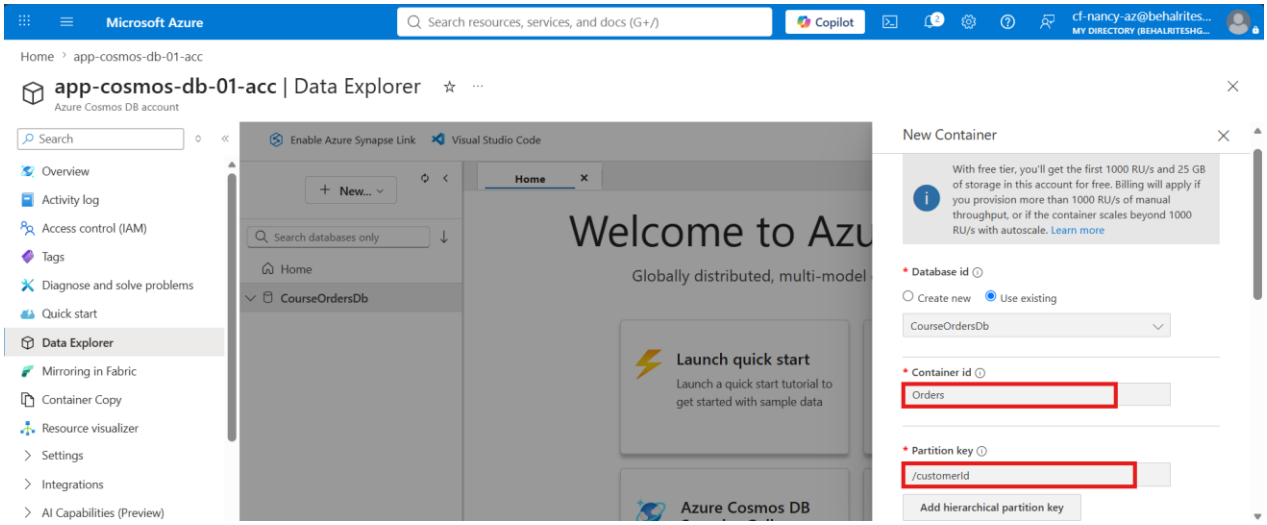
- Click **OK**.
- Verify that the database is created successfully.
- This step creates a database to organize containers and documents.

Step 3 – Create a Container

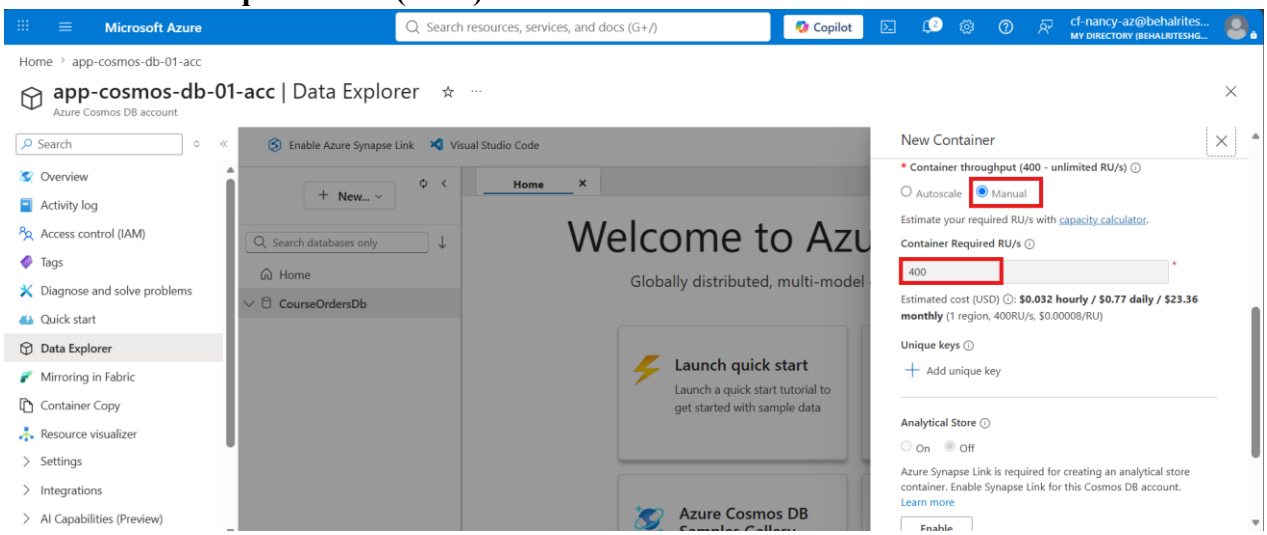
- Expand **CosmosOrdersDB**.



- Click **New Container**.
- Configure the following settings:
  - **Container ID:** Orders
  - **Partition Key:** /customerId



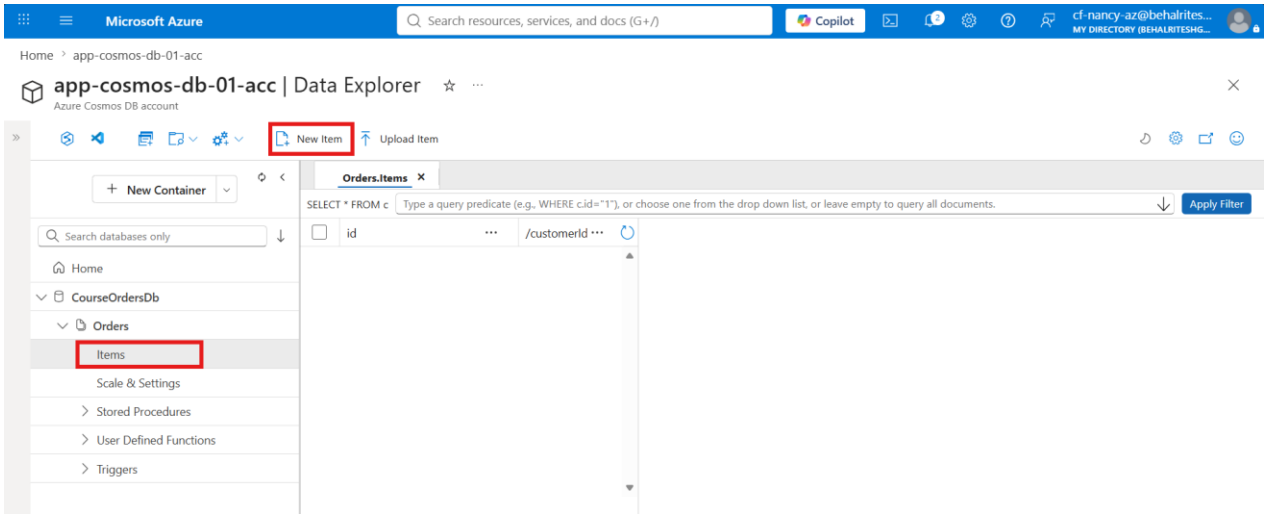
- **Container Throughput:** Manual
- **Request Units (RU/s):** 400



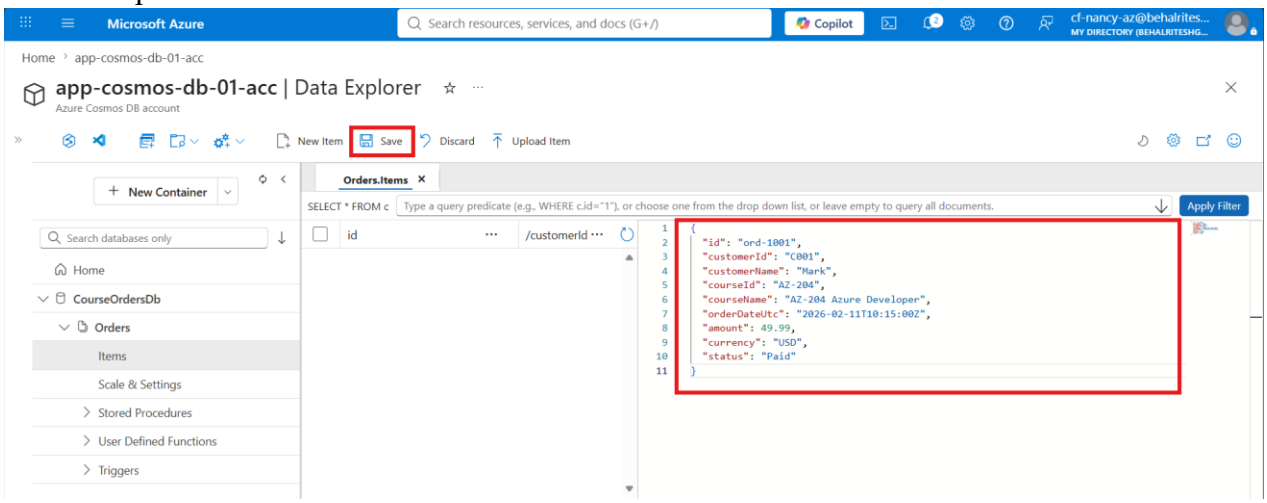
- Click **OK**.
- Verify that the **Orders** container is created.
- This step creates a container that stores JSON documents.

Step 4 – Create the First Document

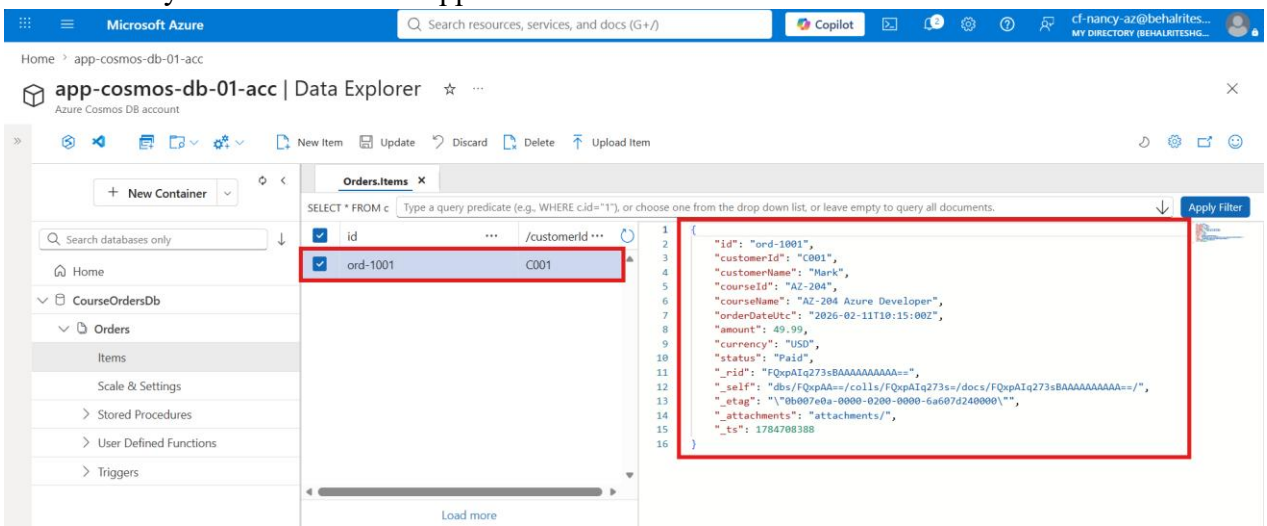
- Expand the **Orders** container.
- Select **Items**.



- Click **New Item**.
- Replace the default JSON with the first order document.



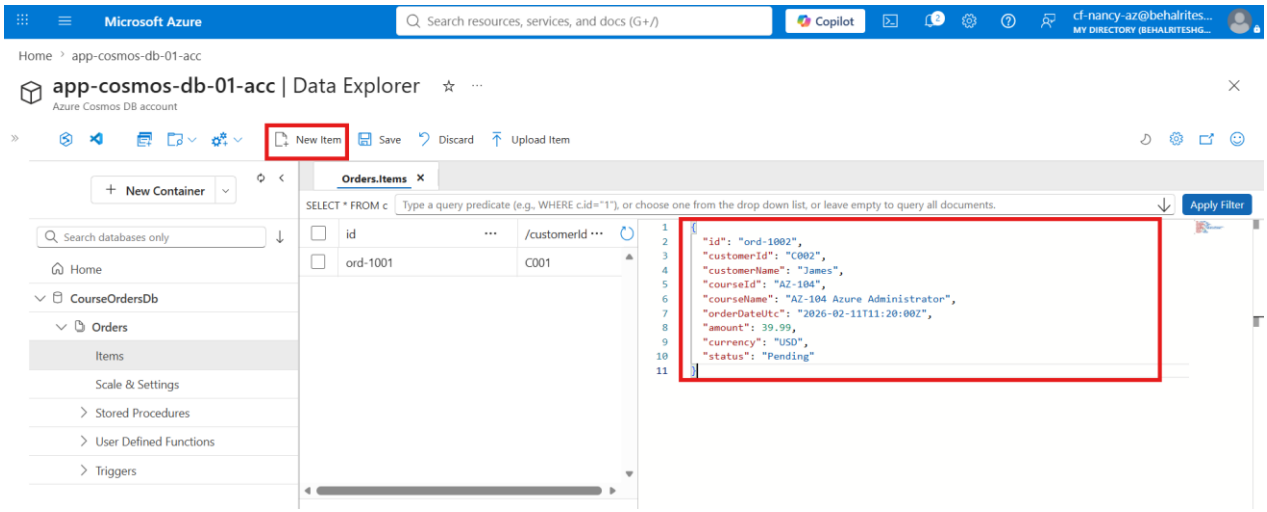
- Click **Save**.
- Verify that the document appears under **Items**.



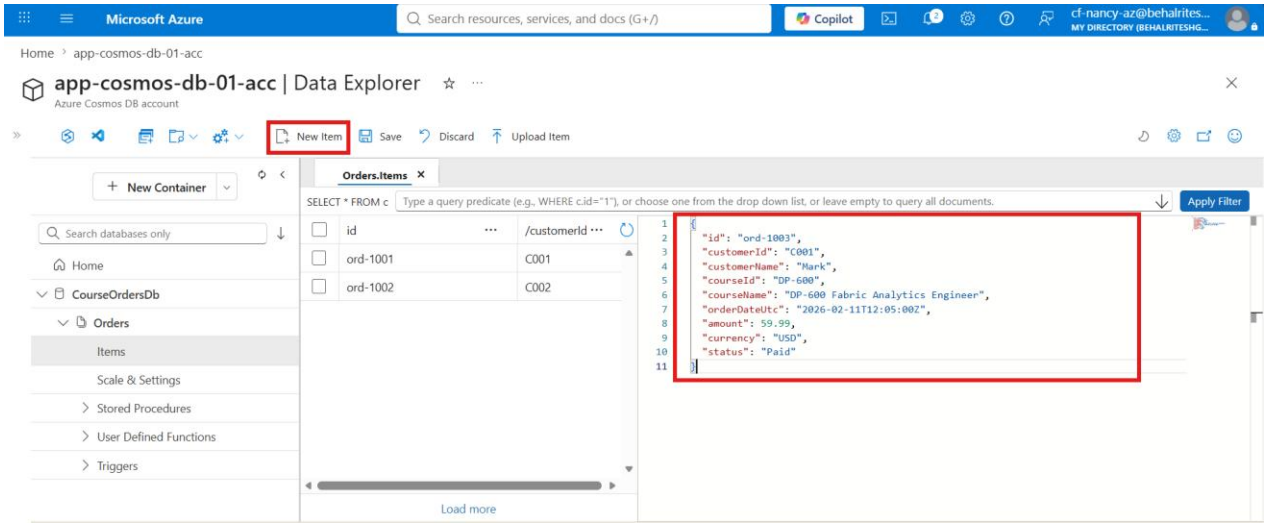
- Observe that Azure Cosmos DB automatically adds system-generated properties to the document.
- This step stores the first JSON document in the container.

Step 5 – Create Additional Documents

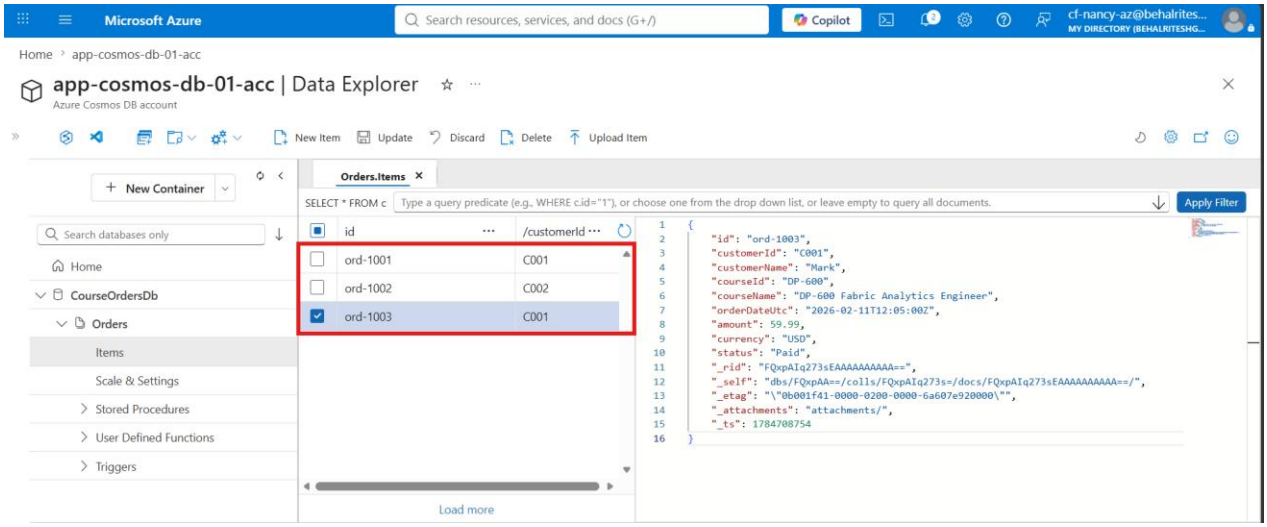
- Click **New Item**.
- Add the second order document.



- Click **Save**.
- Repeat the process to add the third order document.



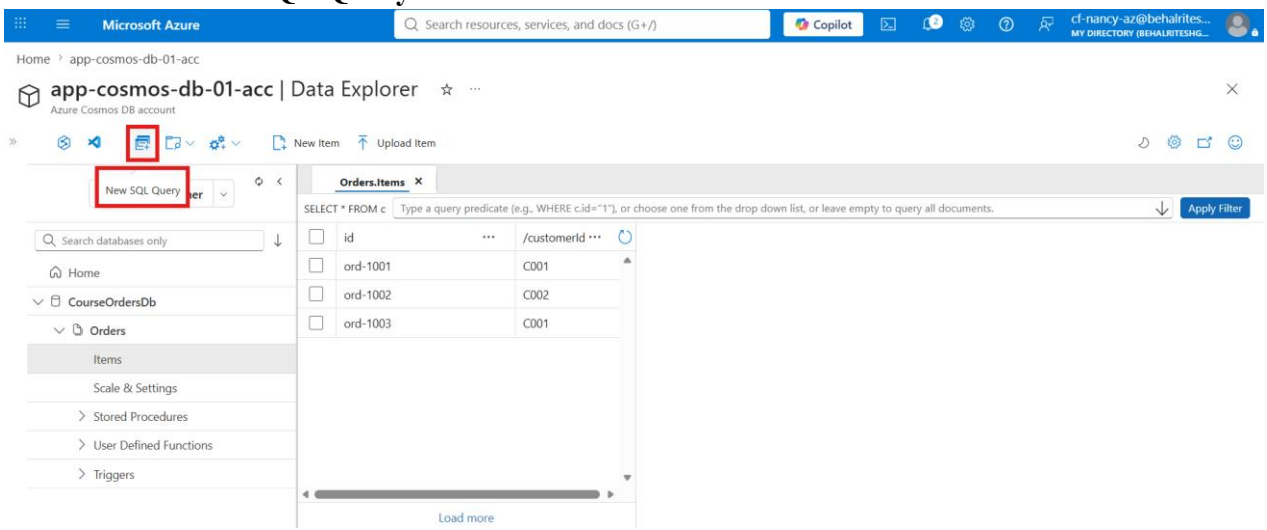
- Verify that all three documents are displayed under **Items**.



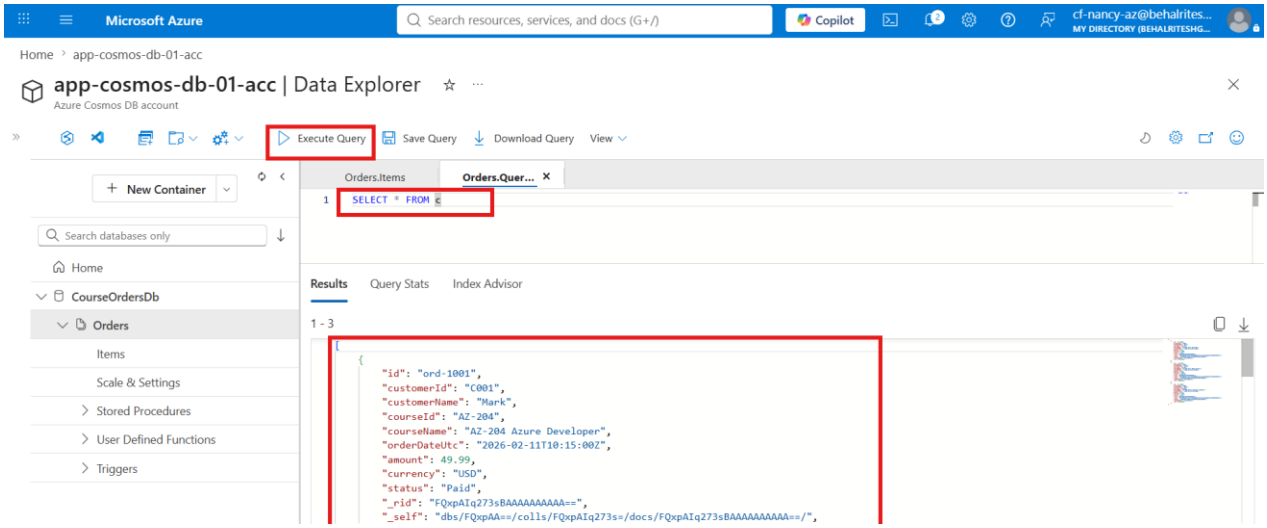
- Observe that documents having the same **CustomerId** are grouped using the configured partition key.
- This step stores multiple documents and demonstrates data partitioning in Azure Cosmos DB.

Step 6 – Create a SQL Query

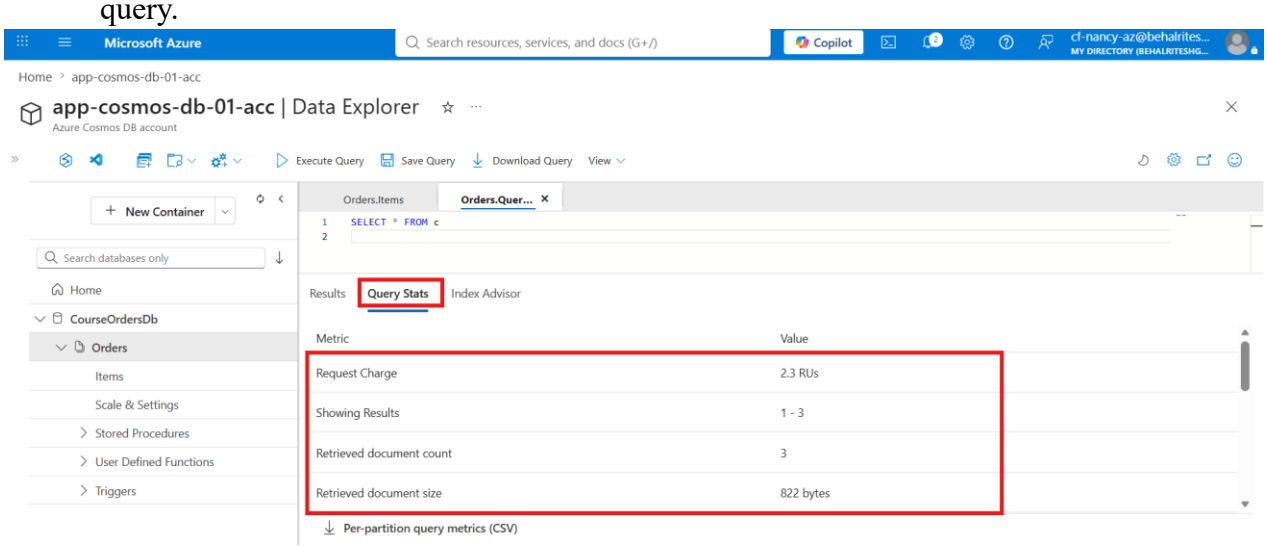
- Select **New SQL Query**.



- Enter the following query:
  SELECT * FROM c



- Click **Execute Query**.
- Verify that all items stored in the container are displayed.
- Observe the **Query Stats** section to view the Request Units (RU) consumed by the query.

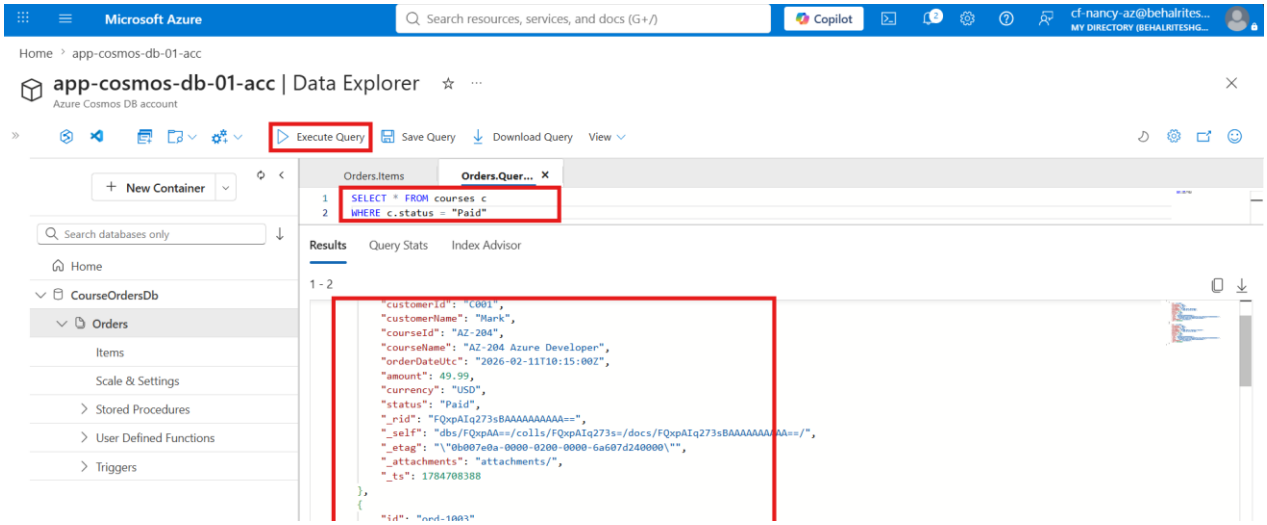


- This step retrieves all documents stored in the container using a SQL query.

Step 6 – Filter Items Using a WHERE Clause

- Modify the query to filter documents by category or another field available in your data.
- Run:

```
SELECT * FROM c
WHERE c.status = "Paid"
```

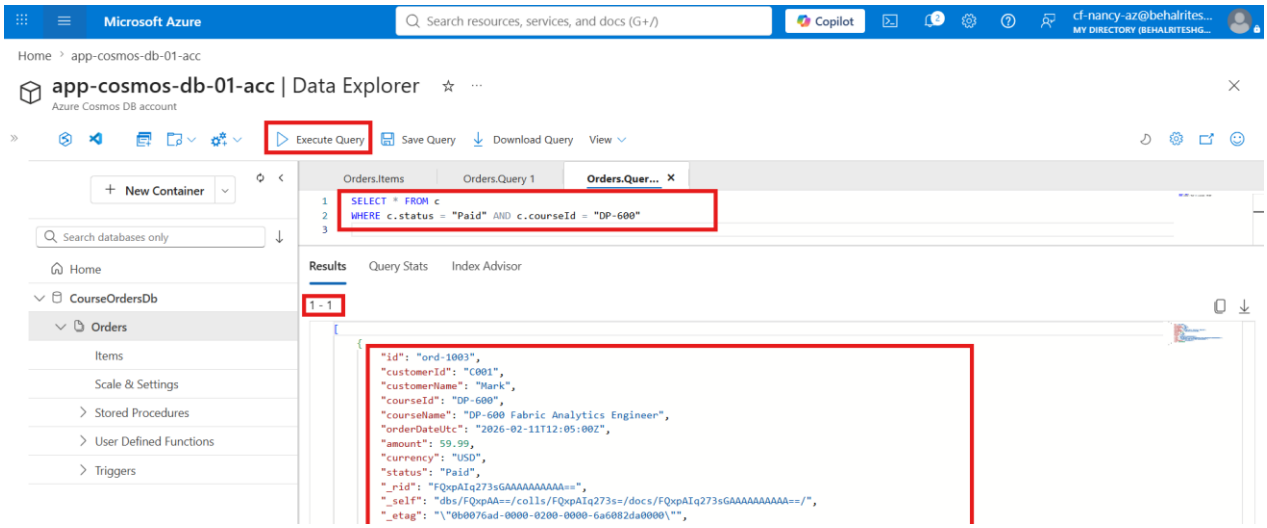


- Click **Execute Query**.
- Verify that only the matching documents are returned.
- This step demonstrates how to filter documents using the **WHERE** clause.

Step 7 – Use Multiple Filter Conditions

- Update the query to include multiple conditions.
- Run:

```
SELECT * FROM c
WHERE c.status = "Paid" AND c.courseId = "DP-600"
```

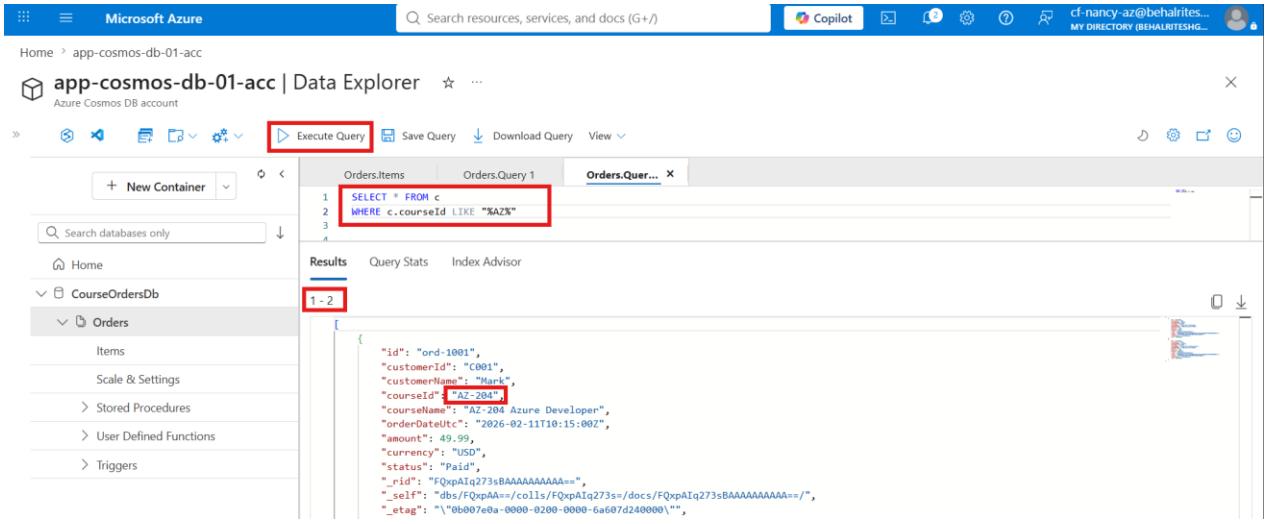


- Click **Execute Query**.
- Verify that only the documents satisfying both conditions are displayed.
- This step demonstrates how to filter data using multiple conditions with the **AND** operator.

Step 8 – Search Using the LIKE Operator

- Create a query to search for items containing specific text.

```
SELECT * FROM c
WHERE c.Name LIKE "%AZ%"
```



- Click **Execute Query**.
- Verify that only the matching documents are returned.
- This step demonstrates how to search documents using the **LIKE** operator.

Verification

Verify the following:

- The Azure Cosmos DB account, database, and container are created successfully.
- Multiple JSON documents are stored in the Orders container.
- SQL queries return the expected documents.
- Filtered queries display only matching records.
- The LIKE query successfully searches documents based on the specified text.